

CNN Showcase: Image Classification and Model Explainer

1 What is Machine Learning?

Machine learning is a subfield of artificial intelligence concerned with the development of computational systems that improve their performance on a given task through exposure to data, rather than through explicit, rule-based programming. Instead of a programmer writing down every rule the computer should follow, a machine learning model is shown many examples and gradually learns the underlying patterns for itself. This is a fundamentally different way of building software: rather than encoding a fixed set of instructions, the developer's role shifts towards choosing a suitable model, gathering good-quality data, and designing a process by which the model can learn effectively.

At the heart of every machine learning model is a set of internal numbers, often called *parameters* or *weights*, which determine exactly how the model turns an input (such as an image) into an output (such as a predicted category). When a model is first created, these numbers are typically set randomly, meaning its initial predictions are essentially meaningless guesses. Training is the process of gradually adjusting these numbers, example by example, until the model's predictions become reliably useful. Critically, the same underlying model architecture can be trained on entirely different datasets to solve entirely different problems so it is the training data, not the code itself, that ultimately shapes what a given model becomes capable of doing.

Definition 1 (Training). The process by which a model repeatedly looks at examples, checks how wrong its guesses are, and slightly adjusts itself to be less wrong next time.

Machine learning is usually grouped into a small number of broad approaches, distinguished mainly by what kind of data the model learns from and what feedback, if any, it receives. *Supervised learning*, discussed in more detail below, trains a model using examples that already have a known correct answer attached. *Unsupervised learning*, by contrast, works with data that has no attached answers at all, and instead asks the model to find structure or groupings within the data on its own. A third approach, *reinforcement learning*, trains a model by letting it take actions within an environment and rewarding or penalising it based on the outcomes, rather than showing it correct answers directly. The model demonstrated in this showcase falls into the first of these categories.

A useful, everyday analogy: think of how a child learns to recognise a dog. Nobody hands them a rulebook of exact measurements; they simply see many dogs of different breeds, sizes and colours, and over time learn to generalise what "dog" means. Machine learning models learn in a broadly similar way, just using data and mathematics instead of intuition. The ultimate goal of training is not for a model to memorise its examples perfectly, but for it to *generalise* by to correctly handle new examples it has never encountered before, in much the same way a child can recognise a dog breed they have never personally seen.

1.1 Supervised Learning

Machine learning comes in several flavours, but the model behind this demo uses the most common approach for tasks like this one: *supervised learning*, in which every example the model is shown during training comes with a correct answer already attached, called a *label* – in this project, every training image was a photo of a face that had already been sorted into one of four categories (Boys, Girls, Man, or Woman), and the model's job during training was simply to get better and better at guessing the correct label on its own, checking itself against the true label each time and adjusting accordingly, which is "supervised" in the sense that the correct answers act as a supervisor, correcting the model's mistakes throughout the process.

Definition 2 (Label). The correct answer attached to a training example, which the model is trying to learn to predict on its own for new, unlabelled examples.

1.2 What is a Convolutional Neural Network (CNN)?

A Convolutional Neural Network (CNN) is a type of deep learning algorithm that is particularly effective for analysing visual data. It is designed to automatically and adaptively learn spatial hierarchies of features from input images, meaning it starts by noticing very simple patterns and builds up to recognising complex objects. CNNs are widely used in image classification, object detection, and other computer vision tasks, and are the type of model used in this showcase.

CNNs were specifically designed to overcome a weakness of simpler neural networks when applied to images. A standard, fully-connected network treats every individual pixel as a completely independent input, with no inherent notion that two neighbouring pixels are related to one another. This throws away one of the most important properties of an image: that meaningful patterns (an edge, an eye, a jawline) are formed from pixels that are spatially close together. Convolutional layers preserve this spatial structure by examining small, local neighbourhoods of pixels at a time, rather than the entire image at once.

Mathematically, a convolutional layer works by sliding a small matrix of learnable numbers, called a *filter* or *kernel*, across the input image, and at each position computing the sum of the element-wise products between the filter and the patch of the image it currently covers. For a filter K of size $k \times k$ applied to an image I , the value of the resulting feature map at position (i, j) is given by:

$$S(i, j) = (I * K)(i, j) = \sum_{m=0}^{k-1} \sum_{n=0}^{k-1} I(i + m, j + n) \cdot K(m, n) \quad (1)$$

In practice, a learnable *bias* term b is added to this sum, and the result is passed through a non-linear *activation function* f , most commonly the Rectified Linear Unit (ReLU) used throughout this project's network, defined simply as:

$$f(x) = \max(0, x) \quad (2)$$

Definition 3 (Activation Function). A small mathematical function applied after each layer that decides how strongly a neuron "fires" in response to what it has seen, allowing the network to represent more complex, non-linear relationships than simple addition and multiplication alone could.

This gives the full output of a single convolutional filter at position (i, j) as:

$$Y(i, j) = f(S(i, j) + b) \quad (3)$$

A single convolutional layer typically contains many such filters, 16, 32, or 64 in the network used for this project, each learning to detect a different pattern, and each producing its own feature map. Finally, once an image has passed through all of the network's convolutional and pooling layers and reached the output layer, the model needs to convert its four raw output scores into something more interpretable: a confidence, expressed as a percentage, for each category. This is done using the *softmax function*, which converts a list of raw scores $z_1, \dots, z_C$ (one per category, $C = 4$ in this project) into values between 0 and 1 that sum to exactly 1:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_{j=1}^C e^{z_j}} \quad (4)$$

The category with the highest resulting value is taken as the model's prediction, and that value, expressed as a percentage, is what this application displays as its confidence.

1.3 How does a CNN work?

Having seen the mathematics behind a single convolutional filter, it is worth stepping back and looking at how an entire CNN is built up out of many such filters, arranged into stages, to go from a raw photograph to a final decision.

Convolutional layers. As described above, each convolutional layer applies many filters to its input, each producing its own feature map. Early in the network, these filters tend to respond to very simple, low-level patterns, a vertical edge, a patch of a particular colour, the boundary between two textures. None of these filters are hand-designed by a person; during training, the network starts with random filters and gradually adjusts them, over many examples, until they become useful at telling the categories apart.

Definition 4 (Feature Map). The output produced when a filter is slid across an image, highlighting where in the image that filter's pattern was found.

Pooling layers. After a convolutional layer, a CNN typically applies a pooling layer, which shrinks each feature map down by summarising small regions of it. The most common approach, *max pooling*, simply keeps the largest value found within each small window and discards the rest. For a pooling window $R_{i,j}$ (commonly a 2×2 region) taken from a feature map Y , the pooled output is:

$$P(i, j) = \max_{(m, n) \in R_{i, j}} Y(m, n) \quad (5)$$

This serves two purposes: it makes the network considerably faster and lighter to run, since there is simply less data to carry forward to the next layer, and it makes the network somewhat tolerant of a detected feature shifting by a few pixels, an eye that is slightly further left in one photo than another should still be recognised as an eye.

Stacking layers. A single convolution-and-pooling stage on its own can only detect fairly simple patterns. The real strength of a CNN comes from stacking many such stages on top of one another, so that the network builds understanding hierarchically. Concretely, for the model used in this showcase: the earliest layers respond to simple visual patterns such as edges around the jawline or the boundary between skin and hair; the next layers combine these simple patterns into more recognisable structures, such as the relative shape and spacing of facial features; and by the final convolutional layers, the network is no longer working with anything resembling raw pixels at all, but with a highly compressed summary of "what kind of face is this", built entirely out of the patterns detected earlier in the network. Crucially, no person told the network to specifically look for jawlines or eye spacing, these are simply the patterns that training found to be useful for telling the four categories apart.

Fully connected layers. Once an image has passed through every convolutional and pooling stage, the network flattens its final feature maps into a single list of numbers and passes them through one or more fully connected (**Dense**) layers, which combine everything detected so far into a final decision. As described in the previous subsection, the very last layer produces one raw score per category, and the softmax function converts these into the confidence percentages this application displays on screen – the category with the highest resulting value becomes the model’s prediction.

2 How was this model trained?

2.1 What is TensorFlow?

TensorFlow is an open-source machine learning framework originally developed by the Google Brain team and released publicly in 2015. It provides a comprehensive ecosystem for building and deploying machine learning models, including deep learning architectures like CNNs, and has since become one of the most widely used machine learning frameworks in both industry and research.

At its core, TensorFlow handles the underlying mathematics of a neural network, representing the layers, filters, and weights described in the previous section as structured, multi-dimensional arrays of numbers called *tensors*, and efficiently computing how those numbers should change during training. Rather than requiring a developer to implement operations like convolution or backpropagation (the process by which a model works out how to adjust its weights after a mistake) from scratch, TensorFlow provides these as ready-made, heavily optimised building blocks.

Much of this project was built using *Keras*, TensorFlow’s official high-level API, accessed here through `tensorflow.keras`. Where TensorFlow itself operates at a fairly low level, Keras provides a much simpler, more readable interface for assembling a network layer by layer, exactly the **Sequential** model structure described in the training section of this document, while still running on top of TensorFlow’s underlying engine.

Another major advantage of TensorFlow is its flexibility around hardware. The same TensorFlow code can run on an ordinary CPU, be automatically accelerated using a GPU when one is available, or even run on specialised hardware called TPUs (Tensor Processing Units) designed by Google specifically for machine learning workloads. TensorFlow also extends well beyond training: as covered later in this document, its surrounding ecosystem includes TensorFlow Lite, used to deploy this project’s model efficiently for the live webcam demo, alongside other tools for deploying models to web browsers and production servers.

2.2 What dataset was used to train this model?

The dataset used to train this model is the "All-Age-Faces Dataset" created by Tsinghua University of Science and Technology, which contains a diverse collection of facial images across different age groups. The dataset includes images of men, women, boys, and girls, allowing the model to learn and classify faces based on age and gender.

2.3 Preparing the images

Before any training can begin, the raw photographs need to be put into a consistent, predictable form that the network can work with. This project’s preparation pipeline involved a few steps:

- **Resizing.** Every image, regardless of its original size, was resized to a fixed 180×180 pixels, and processed in batches of 32 images at a time. A network expects every input to be the same shape, so this step is essential before anything else can happen.
- **Splitting the data.** The dataset was split so that 80% of the images were used for training and the remaining 20% were held back as the validation set described earlier, used only to check the model’s performance on faces it had not trained on.
- **Normalising pixel values.** Digital images are usually stored with colour values ranging from 0 to 255. A *Rescaling* layer was used to compress this range down to between 0 and 1. Neural networks generally train more smoothly and reliably on these smaller, more consistent numbers.
- **Performance optimisation.** Three additional techniques were applied purely to make training run efficiently: *caching* keeps images in memory after they are first read from disk so they don’t need to be reloaded every epoch; *prefetching* allows the next batch of images to be prepared while the current batch is still being processed by the model, rather than the two happening strictly one after another; and *shuffling* randomly reorders the images so the model doesn’t accidentally learn something from the order the files happened to be stored in.

2.4 Building the network

With the data prepared, the network itself was assembled using Keras’s **Sequential** model, which simply means the data flows through each layer one after another, in a fixed order, applying the convolution, pooling, and fully-connected principles described in the CNN section above. Concretely, this project’s architecture consists of three convolutional blocks (each pairing a **Conv2D** layer with a **MaxPooling2D** layer) using 16, 32, and 64 filters respectively – an increasing filter count at each stage being a common design choice, reflecting that later layers need to represent a wider variety of more complex, combined patterns than earlier layers do – followed by a **Flatten** layer, a 128-neuron **Dense** layer with a ReLU activation, and a final **Dense** output layer with one neuron per category.

Once assembled, the model was *compiled* – configured with an *optimizer*, which controls exactly how the network’s internal numbers are adjusted after each mistake (this project used a popular, well-performing optimizer called **Adam**), and a *loss function*, which measures how wrong a given prediction was (this project used **Sparse Categorical Crossentropy**, a standard choice for problems with more than two categories).

2.5 The training process

The model was trained using TensorFlow/Keras over multiple passes (*epochs*) through the dataset. On each pass, the model made predictions on a batch of training images, measured how far off those predictions were using the loss function described above, and nudged its internal filters slightly in the direction that would reduce that error next time.

A portion of the dataset was deliberately held back from training, known as the *validation set*, and used only to check how well the model performed on faces it had never seen before. This distinction matters: a model can appear to do very well simply by memorising the training images, without actually learning to generalise. This is known as *overfitting*.

Definition 5 (Overfitting). When a model becomes very good at the exact examples it was trained on, but performs poorly on new, unseen examples – similar to a student memorising practice questions instead of understanding the underlying topic.

To help the model generalise better rather than simply memorise, two changes were made together for the final version of the model. First, *data augmentation* was applied during training – automatically creating slightly modified versions of training images (horizontal flips, small rotations, and slight zooms) so the model sees more variety without needing more raw data. Second, a *Dropout* layer was added directly into the network, positioned just before the final decision-making layers.

Definition 6 (Dropout). A layer that randomly switches off a fraction of the network’s neurons during each training step, forcing the remaining neurons to not over-rely on any single feature. This makes the network more robust, similar to how a sports team that trains without always relying on its one star player tends to perform better when that player is unavailable.

With a 20% dropout rate added alongside data augmentation, the model reached a validation accuracy of approximately **77%** across the four categories, with training and validation accuracy tracking much more closely together than in the earlier, unregularised attempt.

It is worth noting that some categories are inherently harder to separate than others: distinguishing an adult man from a teenage boy, for example, is a genuinely subjective judgement even for a human observer working from a single photo, so some level of confusion between closely related categories is expected rather than a flaw in the model.

2.6 Measuring success: what is accuracy?

Throughout training, the model’s performance was tracked using *accuracy*: the percentage of images the model classifies correctly.

Definition 7 (Validation Accuracy). The accuracy a model achieves on the held-out validation set – images it never saw during training – used as an honest estimate of how well it will perform on new, real-world images.

It is important to track *validation* accuracy specifically, rather than accuracy on the training data alone. A model can often be pushed to near-perfect accuracy on the exact images it was trained on simply by memorising them, without having learned anything generalisable – this was, in fact, observed during an earlier training attempt for this project, where training accuracy climbed to around 98% while validation accuracy stalled and then worsened, a textbook example of the overfitting described above, and the reason the data augmentation and Dropout changes described earlier were introduced.

3 How was this model deployed? (Frontend)

A trained model is only useful once it can be used outside of the training environment. This section covers how the model shown in this demo was taken from a training notebook to the live application in front of you.

3.1 From training to a deployable model

A trained Keras model can be exported in several different formats, each suited to a different situation. Four were considered for this project:

- **SavedModel** – TensorFlow’s standard, most portable export format, intended for serving a model in production environments such as a cloud platform or a dedicated model server.
- **.keras (native Keras format)** – the modern, general-purpose “save my progress” format, ideal for picking up exactly where training left off, whether to keep training further or to make predictions in Python.
- **HDF5 (.h5)** – an older single-file format that serves a similar purpose to .keras. It is still widely supported and commonly seen in existing tutorials, but has largely been superseded by the native Keras format for new projects.
- **TensorFlow Lite (.tflite)** – a specialised, compressed format designed for fast, efficient predictions on a single device, such as a phone, an embedded system, or an ordinary laptop, rather than a large-scale training environment.

For this application, **TensorFlow Lite** was the clear choice. The live webcam demo needs to run a prediction repeatedly, many times a second, directly on whatever laptop is showing the demo – not on a remote server. TensorFlow Lite’s interpreter is built specifically for this kind of fast, repeated, on-device inference, which is what allows the bounding box and prediction you see on screen to update smoothly in real time.

3.2 The desktop application

The application running this demo is a cross-platform desktop app, built to run on Windows, macOS and Linux alike. It brings together several pieces of open-source software:

- **OpenCV** is used to access the webcam and automatically detect where a face is in the frame, drawing a live bounding box around it.
- **TensorFlow Lite** runs the trained model against the detected face and returns a predicted category with a confidence score.
- **PySide6** builds the graphical interface you are interacting with, including this very PDF viewer.

To make live predictions feel more stable to a viewer, the app does not simply display whatever the model predicted on the very last frame. Instead, it keeps a short rolling history of recent predictions and shows the most agreed-upon result, only committing to a label once the model is reasonably confident – otherwise showing “Uncertain” rather than a shaky guess. This mirrors good practice in real-world machine learning applications, where a model’s raw output is rarely shown to a user without some additional handling around it.